HW4 Data Analysis Report

Rajveer Singh

2026-04-13

Busiest Airports Analysis

This section examines six major airports and compares their passenger traffic from 2020 to 2024. The goal is to show which airports handled the most passengers and how traffic changed over time.

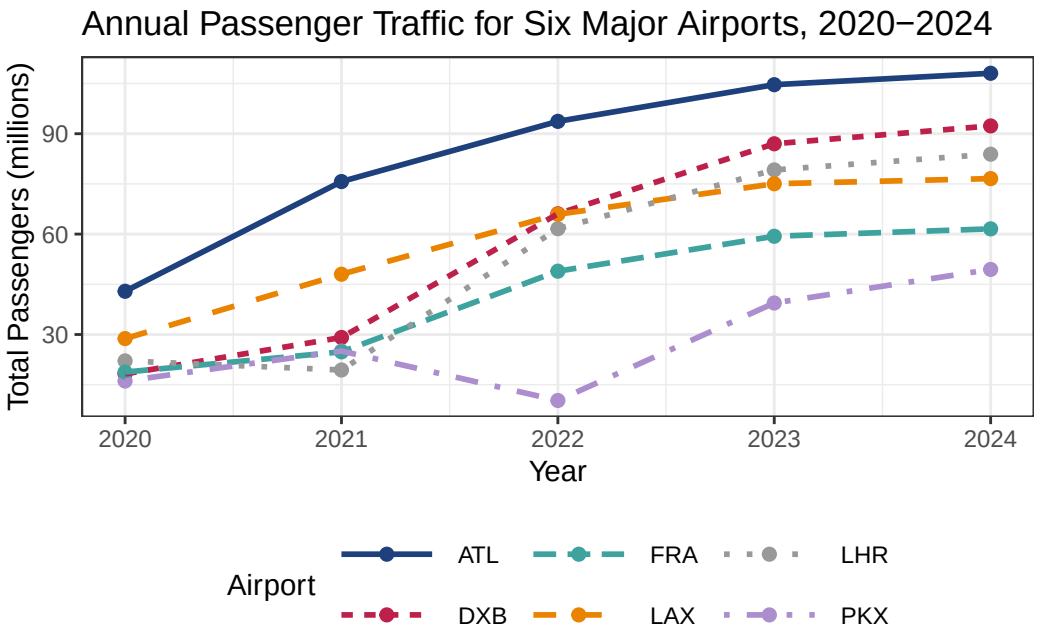
Annual Passenger Totals for Six Major Airports

Passenger counts from 2020 to 2024

Airport	2020	2021	2022	2023	2024
ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766
DXB	18,229,461	29,110,609	66,069,981	86,994,365	92,331,506
LHR	22,111,469	19,393,145	61,614,508	79,183,364	83,884,572
PKX	16,091,449	25,051,012	10,277,623	39,410,776	49,441,029
FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957
LAX	28,779,527	48,007,284	65,924,298	75,050,875	76,588,028

Table of Busiest Airports

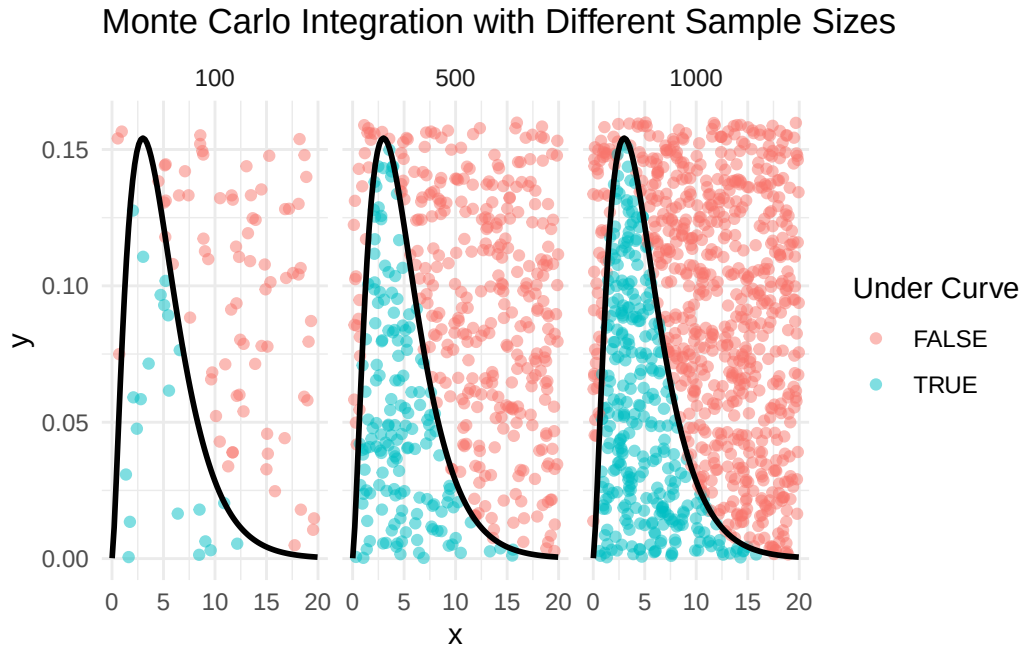
Main Plot



Analysis

The table and line graph show how passenger traffic changed across six major airports from 2020 to 2024. Atlanta remained the busiest airport throughout the period and showed strong growth after 2020. Dubai and London Heathrow also experienced major increases over time, reflecting recovery in air travel. Beijing Daxing shows more fluctuation, while Frankfurt and Los Angeles display steadier trends. ## Monte Carlo Simulation Monte Carlo integration is a numerical method used to estimate the value of an integral using random sampling. Instead of calculating the exact area under a curve directly, the method generates random points and estimates the proportion that fall under the function.

Visualization



Interpretation

The visualization shows that as the number of sampled points increases, the Monte Carlo estimate becomes more stable and accurate. With fewer points, the estimate is less reliable and more variable. As the number of samples increases, the points better approximate the area under the curve, demonstrating how larger sample sizes improve accuracy. # Planning and Prompting GenAI Tools

My Plan

For this task, I planned to analyze the busiest airports dataset and the Monte Carlo simulation from earlier assignments. For the airports dataset, I aimed to ensure the data was in a tidy format with variables such as airport, year, and passenger counts. I also planned to create a clear visualization, such as a line graph, to show how passenger traffic changed over time.

For the Monte Carlo section, I planned to simulate random points and visualize how the estimate of the integral improves with larger sample sizes using a small multiple plot. My goal was to clearly demonstrate how increasing the number of samples affects the accuracy of the approximation. # Reflection

This semester, I learnt that data wrangling is rarely straightforward and requires careful thought about how data is structured before any analysis can begin. Working with the tidyverse family of packages, particularly dplyr and tidyr, helped me understand how to reshape and clean data efficiently using verbs like filter, select, mutate, and pivot_wider.

I also learnt that visualization is not just about making plots, but about choosing the right type of plot to communicate a finding clearly. The busiest airports analysis showed me how a line graph can reveal trends over time in a way that a table alone cannot. Similarly, the Monte Carlo simulation taught me that increasing sample size improves the accuracy of numerical estimates, which I could see visually through the faceted plot.

Working with Quarto this semester was a new experience that changed how I think about reproducibility. I learnt that a good reproducible document should allow anyone to re-run the analysis and get the same results, which requires being deliberate about how code, narrative, and output are organized together. This connects directly to the principles of Open Science, where transparency and reproducibility are essential.

Finally, working with GenAI tools taught me that the quality of the output depends heavily on the quality of the prompt. A detailed, context-rich prompt produces far more useful and specific guidance than a generic one.

Appendix A: GenAI Usage

Tool: ChatGPT (GPT-5.3)
Date: April 13, 2026

Prompt

"Given a dataset of airport passenger counts over multiple years, help me ensure the data is in a tidy format and suggest an appropriate visualization to show trends over time. Also, for a Monte Carlo simulation, suggest how to create a small multiple plot to compare results across different sample sizes."

Response

To organize your airport dataset, you should ensure it follows a tidy format where each row represents a single observation and each column represents a variable. In your case, the dataset should include columns such as Airport, Year, and Passengers.

To visualize trends over time, a line graph is appropriate. You can use a line plot with Year on the x-axis and passenger counts on the y-axis, and group by Airport. This will allow you to clearly compare trends across different airports.

For the Monte Carlo simulation, you can generate random points within a defined region and determine whether they fall under the curve of your function. To compare different sample sizes, you can run the simulation with multiple values of n (such as 100, 500, and 1000).

To visualize this effectively, you can use a small multiple plot by applying faceting. In ggplot2, this can be done using facet_wrap(), which allows you to display separate panels for each sample size. This will clearly show how the approximation improves as the number of samples increases.

Generic Output

The dataset was cleaned by removing missing values and converting variables into appropriate formats. Basic summaries were created to understand the data. A visualization was generated to display relationships between variables, but the approach was general and not specifically tailored to the dataset.

Comparison

The plan-based prompt produced a more relevant and detailed response compared to the generic output. Because the prompt included specific details about the airport dataset and the Monte Carlo simulation, the AI was able to suggest appropriate visualizations such as a line graph and a faceted plot for comparing sample sizes.

In contrast, the generic output was vague and did not provide any specific guidance on how to structure the data or what type of visualization to use. This shows that including clear context and goals in prompts leads to more useful and assignment-specific outputs from GenAI tools.

Appendix B: Code Appendix

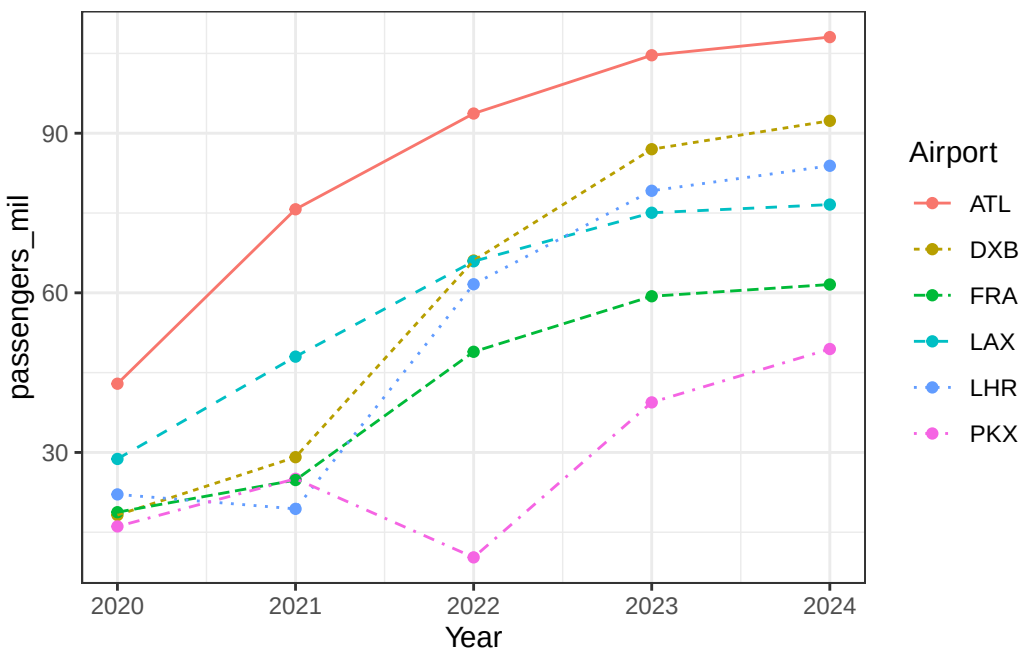
```r
# ========================
# LIBRARIES
# ========================
library(tidyverse)
library(ggplot2)
library(gt)
```

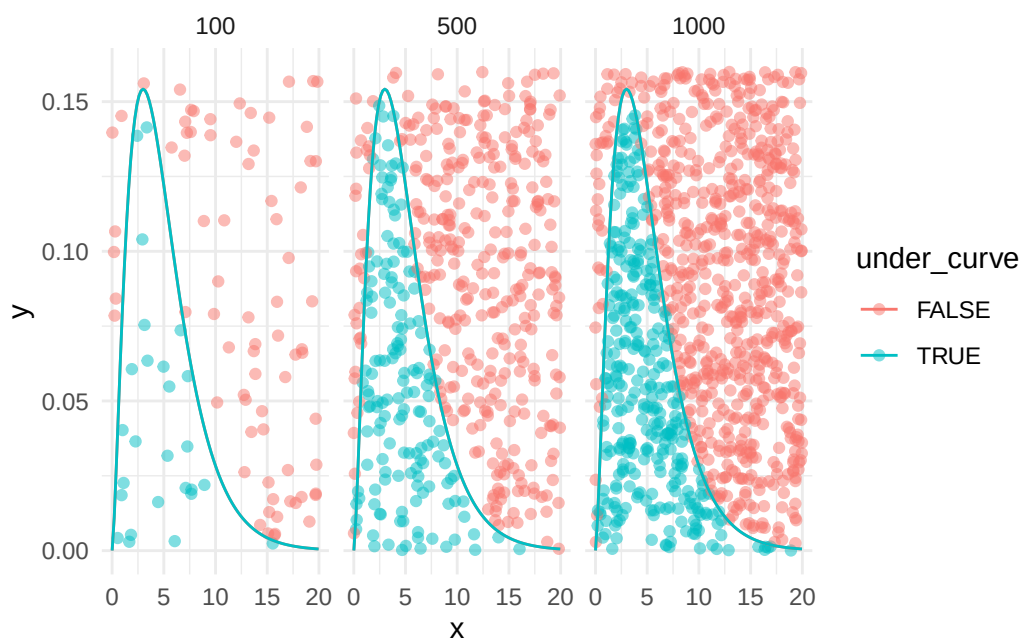
Airport	2020	2021	2022	2023	2024
ATL	42918685	75704760	93699630	104653451	108067766
DXB	18229461	29110609	66069981	86994365	92331506
LHR	22111469	19393145	61614508	79183364	83884572
PKX	16091449	25051012	10277623	39410776	49441029
FRA	18768601	24812849	48918482	59355389	61564957
LAX	28779527	48007284	65924298	75050875	76588028

```r
# ==========================
# BUSIEST AIRPORTS DATA
# ==========================
# Create dataset of six major airports
airport_data <- tibble(
  Airport = rep(c("ATL", "DXB", "LHR", "PKX", "FRA", "LAX"), each = 5),
  Year = rep(2020:2024, times = 6),
  Passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766,
    18229461, 29110609, 66069981, 86994365, 92331506,
    22111469, 19393145, 61614508, 79183364, 83884572,
    16091449, 25051012, 10277623, 39410776, 49441029,
    18768601, 24812849, 48918482, 59355389, 61564957,
    28779527, 48007284, 65924298, 75050875, 76588028
  )
) |>
  mutate(passengers_mil = Passengers / 1000000)

# ==========================
# TABLE CREATION
# ==========================
# Convert to wide format and display as table
airport_data |>
  select(Airport, Year, Passengers) |>
  pivot_wider(names_from = Year, values_from = Passengers) |>
  gt()
```

```r
# ==========================
# AIRPORTS VISUALIZATION
# ==========================
# Create line plot of passenger trends
psuPalette <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300",
```

```
                "#999999", "#AC8DCE")

airport_data |>
  ggplot(aes(
    x = Year,
    y = passengers_mil,
    group = Airport,
    color = Airport,
    linetype = Airport
  )) +
  geom_point() +
  geom_line() +
  theme_bw()
```




```
# ========================
# MONTE CARLO SIMULATION
# ========================
# Function to simulate random points
mc_simulation <- function(n, x_min, x_max, y_min, y_max) {
  tibble(
    x = runif(n, x_min, x_max),
    y = runif(n, y_min, y_max)
```

```r
  ) |>
    mutate(
      f_x = dchisq(x, df = 5),
      under_curve = y <= f_x
    )
}

# Generate data for multiple sample sizes
samples <- c(100, 500, 1000)

mc_data <- map_df(
  samples,
  ~ mc_simulation(.x, 0, 20, 0, 0.16) |> mutate(n = .x)
)

# Plot Monte Carlo simulation
ggplot(mc_data, aes(x = x, y = y, color = under_curve)) +
  geom_point(alpha = 0.5) +
  stat_function(fun = dchisq, args = list(df = 5)) +
  facet_wrap(~ n) +
  theme_minimal()
```